PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Construcción de una API REST con Node.js, Express y MyS

Presentado a: **Instructor Yoli Alexandra Muñoz**

Por Aprendiz: **Sara Isabel Campo Calapsu**

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **08** de **11** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

Contenido

1. Configuración inicial del servidor 3
2. Creación de funciones de prueba 4
3. Desarrollo del módulo de usuarios 8
4. Desarrollo del módulo de productos 13
5. Integracion de rutas y conexión con MySQL 18
6. Identificación y solución de errores 19
7. Pruebas integrales con Postman 22
8. Preguntas de análisis y sustentanción 24
8.1. ¿Qué es Node.js y cuál es su función en el proyecto? 24
8.2. ¿Qué es Express y qué ventajas ofrece? 24
8.3. ¿Qué es una API REST? 24
8.4. ¿Qué diferencia existe entre GET, POST, PUT y DELETE? 25
8.5. ¿Qué función cumple express.json()? 25
8.6. ¿Qué es una clase en JavaScript? 25
8.7. ¿Qué es un objeto y cómo se instancia? 25
8.8. ¿Qué función cumple el constructor? 25
8.9. ¿Qué diferencia existe entre un método tradicional y una propiedad con función flecha? 25
8.10. ¿Para qué se utiliza module.exports? 25
8.11. ¿Qué función cumple require()? 26
8.12. ¿Qué responsabilidad tiene un Controller? 26
8.13. ¿Qué responsabilidad tiene Routes? 26
8.14. ¿Por qué la conexión MySQL se comparte desde app.js? 26
8.15. ¿Qué función cumplen los signos ? en las consultas SQL parametrizadas? 26
8.16. ¿Por qué es importante controlar los errores en una API? 26

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Introducción

En este taller se desarrolló una API REST utilizando Node.js, Express y MySQL, con el propósito de administrar la información de usuarios y productos de una empresa.

Durante el desarrollo se implementaron diferentes rutas y controladores para realizar operaciones de registro, consulta, actualización y eliminación de datos.

También se realizaron pruebas utilizando Postman para verificar el funcionamiento de los endpoints y se identificaron diferentes errores, aplicando sus respectivas soluciones.

1. Configuración inicial del servidor

Se creó un proyecto utilizando Node.js y se instalaron las dependencias necesarias para desarrollar la API.

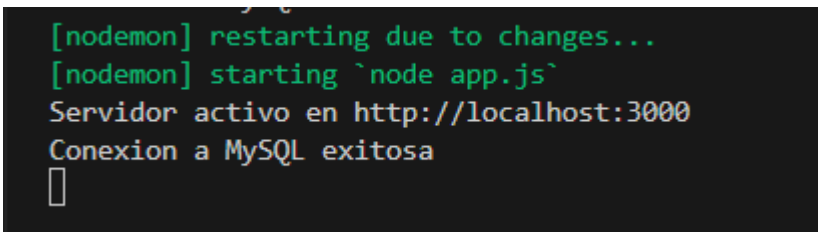
Las herramientas utilizadas fueron:

- Node.js: Que es el que permite ejecutar JavaScript en el servidor.
- Express: Que facilita la creación del servidor y las rutas.
- MySQL2: Me permite conectar el proyecto con la base de datos.
- CORS: Que permite las solicitudes desde otros orígenes.
- Nodemon: Que reinicia automáticamente el servidor cuando se realizan cambios.

Se configuró el archivo app.js para utilizar el puerto 3000, recibir información JSON y establecer la conexión con MySQL.

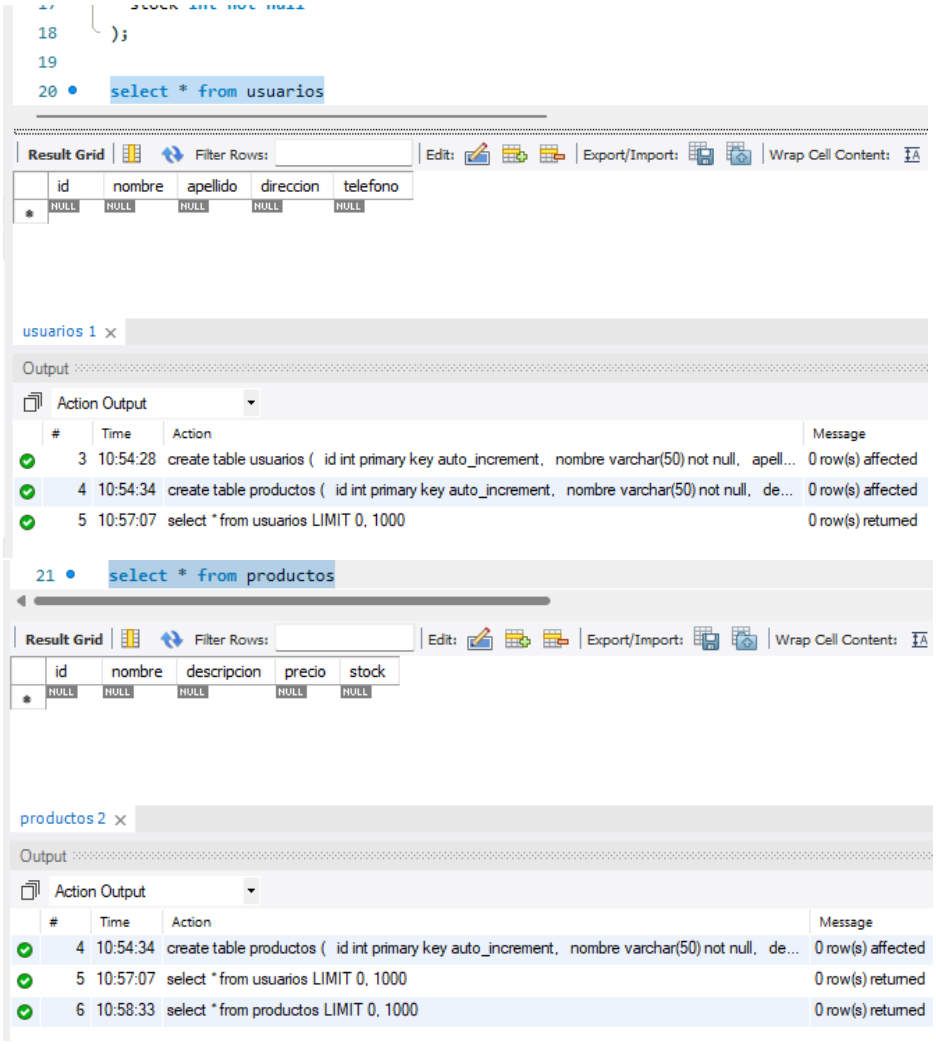
Evidencia 1:

Aquí es donde el servidor se inició correctamente en el puerto 3000 y que se estableció la conexión con la base de datos MySQL.



Evidencia 2:

Luego cree la base de datos llamada taller_api, junto con las tablas usuarios y productos, que son los que permiten almacenar la información correspondiente a cada módulo.



2. Creación de funciones de prueba

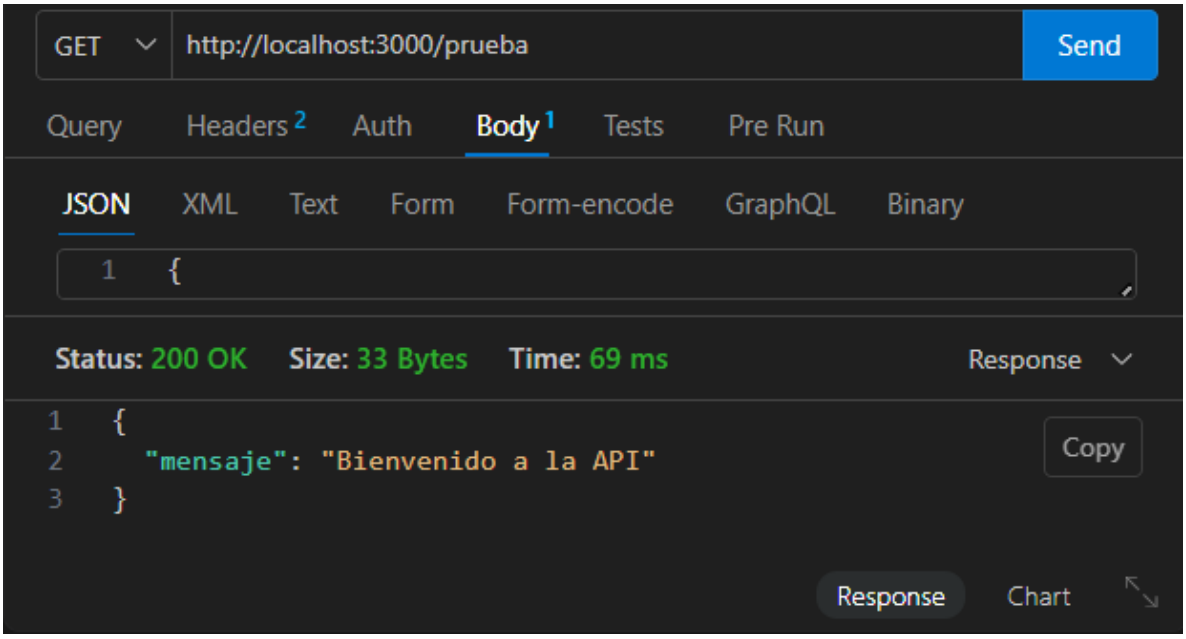
Se desarrolló la clase PruebaController, que contiene cuatro métodos para comprobar que el servidor recibe y procesa correctamente las solicitudes.

Los métodos implementados fueron:

- Bienvenida: Que devuelve un mensaje de bienvenida.
- verificar: Que es el que comprueba que el servidor funciona.
- recibir: Que es el que recibe un nombre y una edad.
- sumar: Quien recibe dos números y devuelve el resultado de la suma.

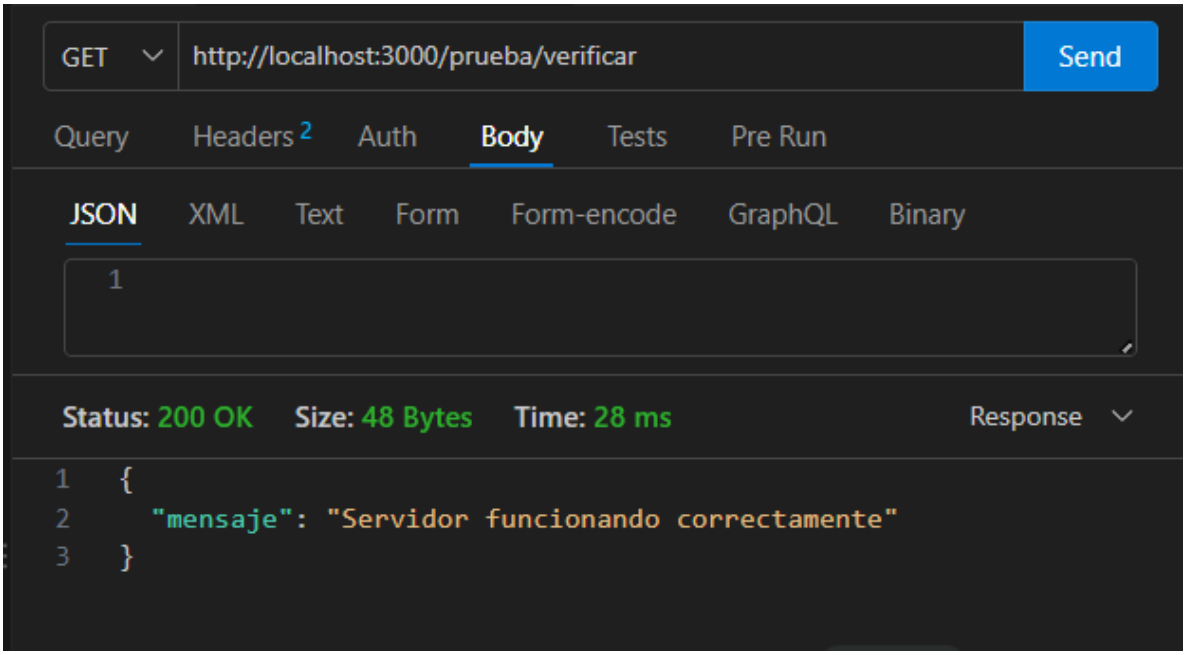
Evidencia 1:

Realice una solicitud GET para comprobar que la API responde correctamente con un mensaje de bienvenida.



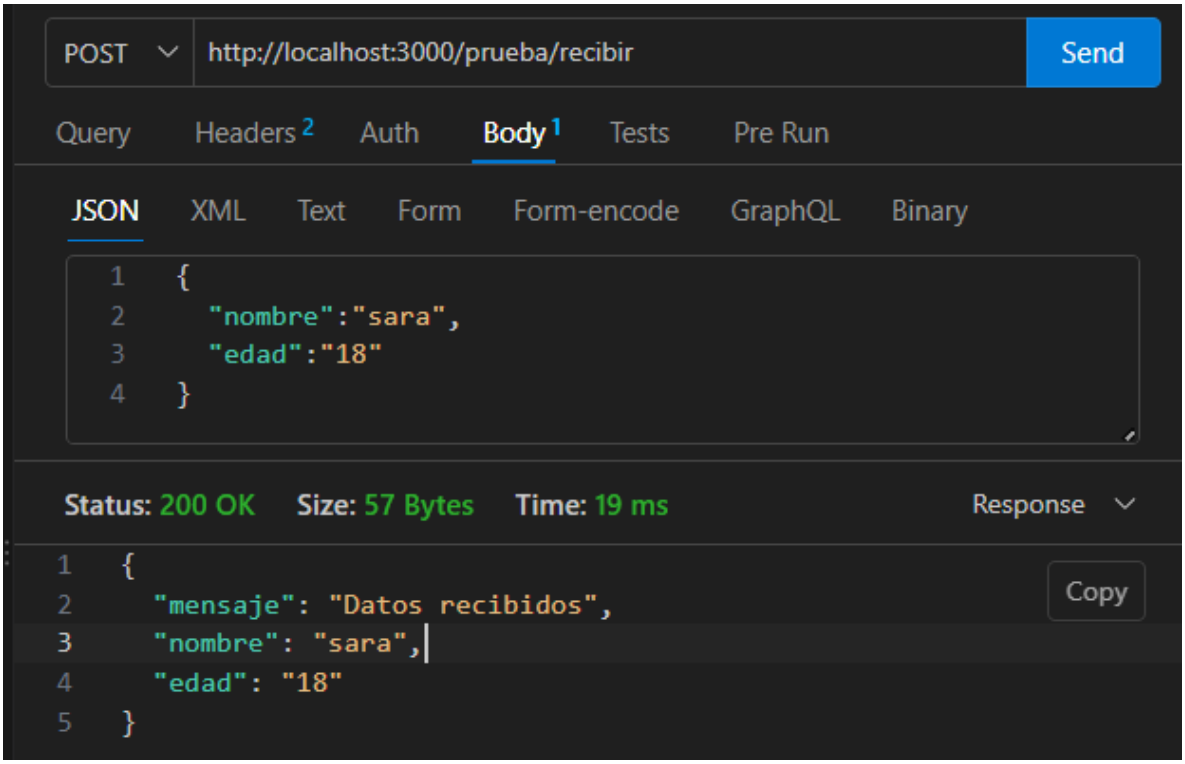
Evidencia 2:

Luego compruebo el funcionamiento del servidor mediante una respuesta que contiene un mensaje y un estado booleano verdadero



Evidencia 3:

Envie los datos de una persona mediante el método POST y el servidor recibió el nombre y la edad y devolvió la información correctamente.



Evidencia 4:

Envie dos números en formato JSON y la API realizó la operación matemática, devolviendo el resultado de la suma.



3. Desarrollo del módulo de usuarios

Se creó la clase UsuariosController para administrar la información de los usuarios almacenados en MySQL.

Se implementaron las siguientes operaciones:

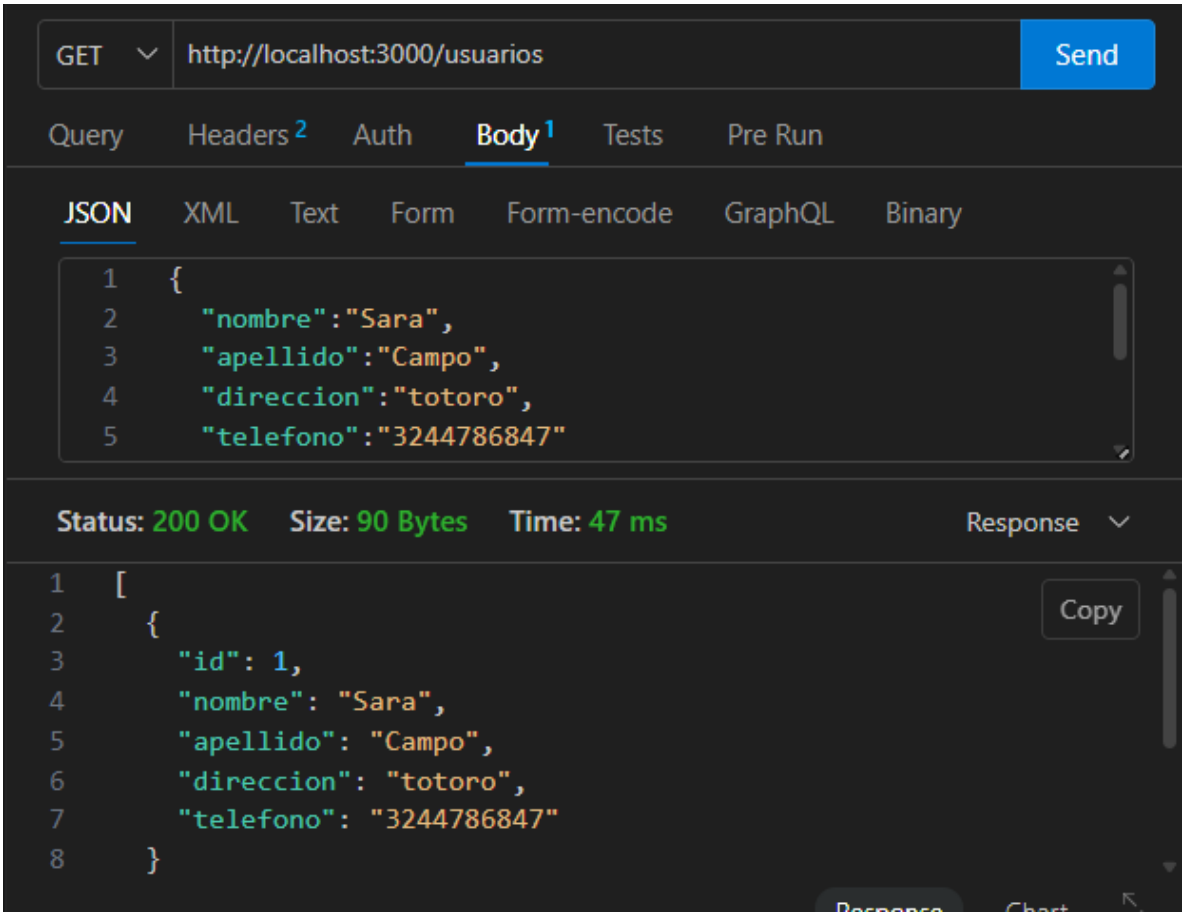
- Guardar: registrar un usuario.
- Traer: consultar todos los usuarios.
- Buscar: consultar un usuario por su identificador.
- Actualizar: modificar los datos de un usuario.
- Eliminar: eliminar un usuario.

Las rutas fueron organizadas en el archivo UsuariosRoutes.js.

Evidencia 1:

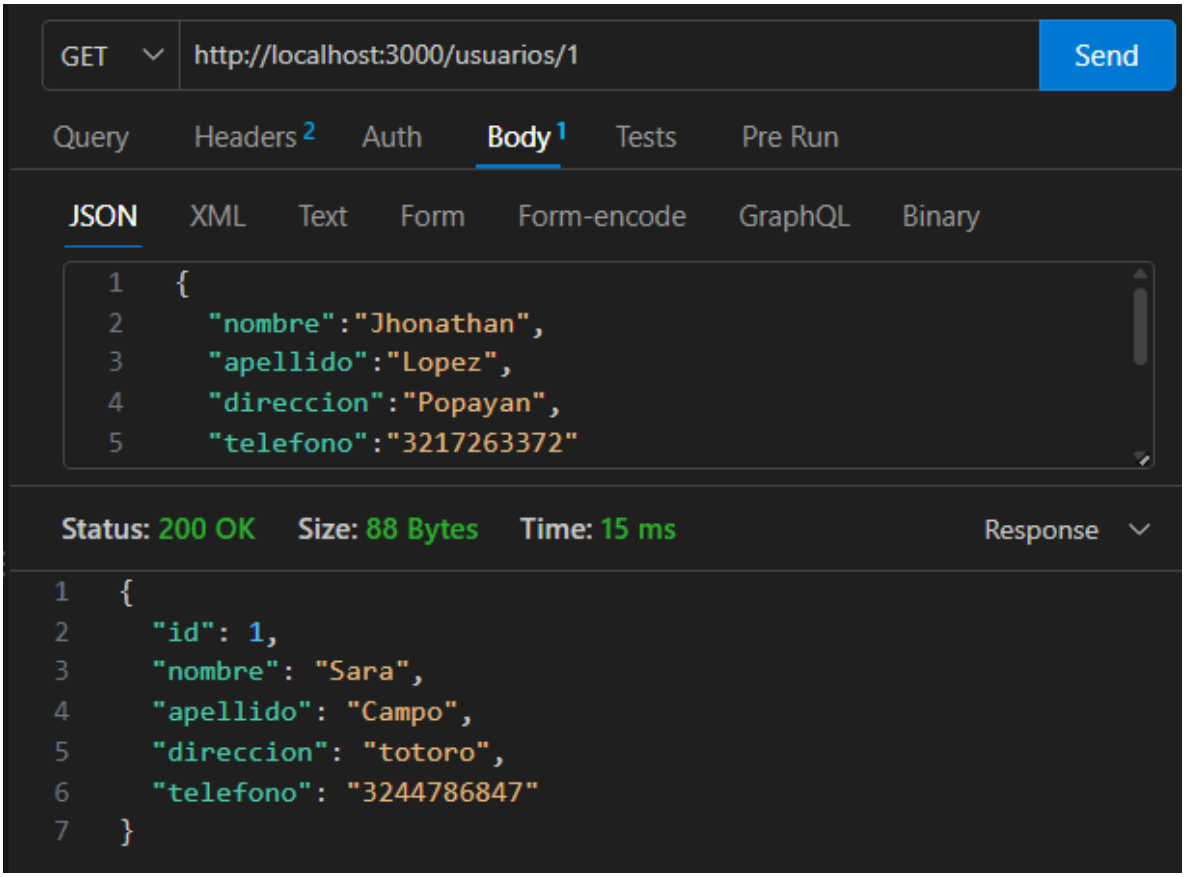
Registre un usuario enviando su nombre, apellido, dirección y teléfono, la API confirma el registro y devuelve el identificador asignado.

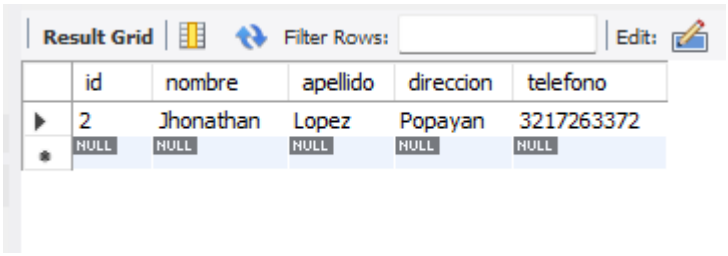
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Evidencia 2:

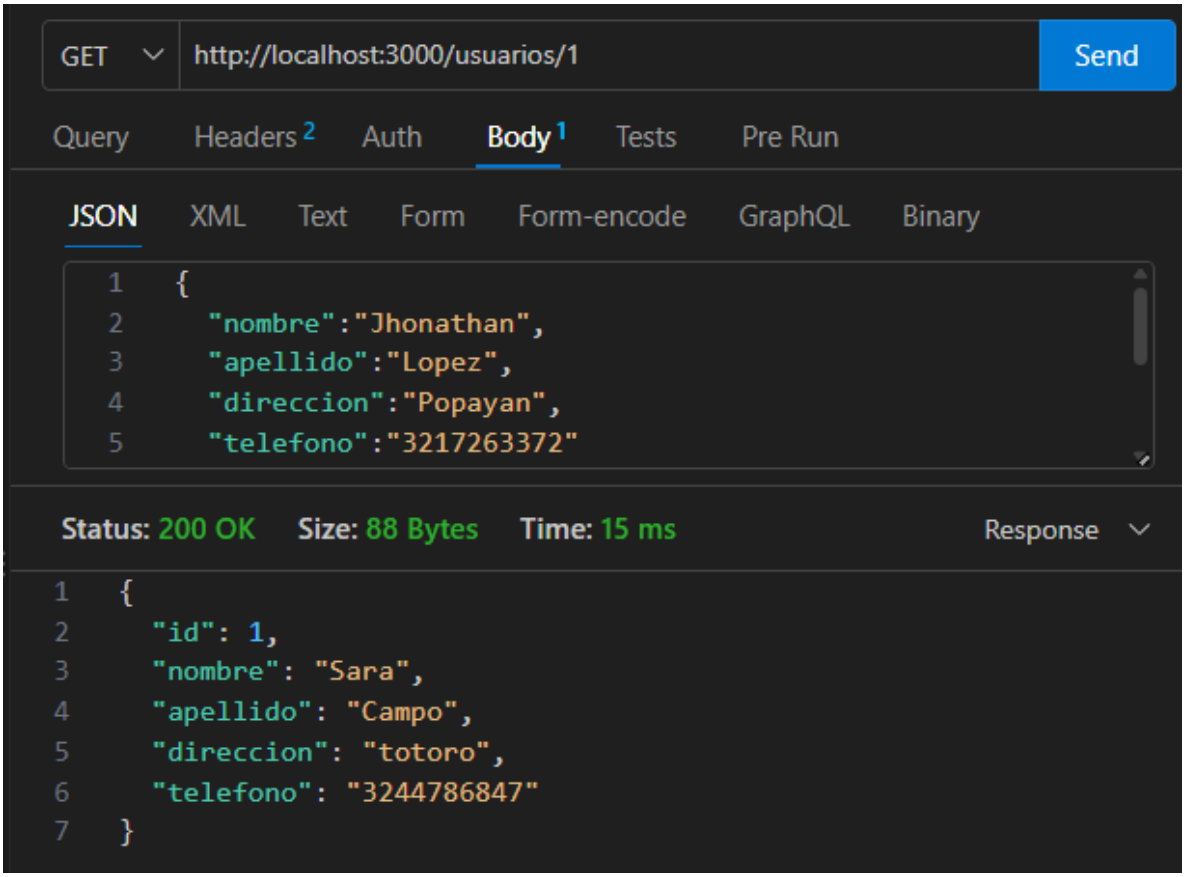
Se consultaron todos los usuarios almacenados en la base de datos y se obtuvo la lista de registros.





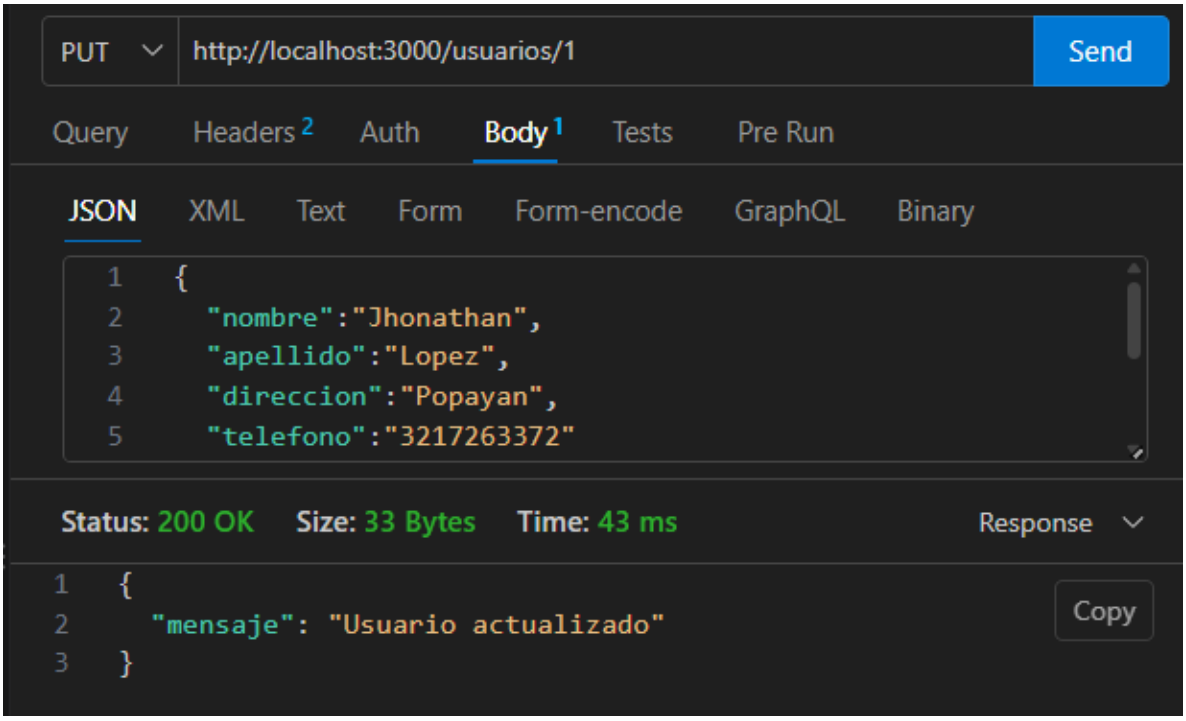
Evidencia 3:

Realice una consulta utilizando el identificador del usuario para obtener únicamente su información.



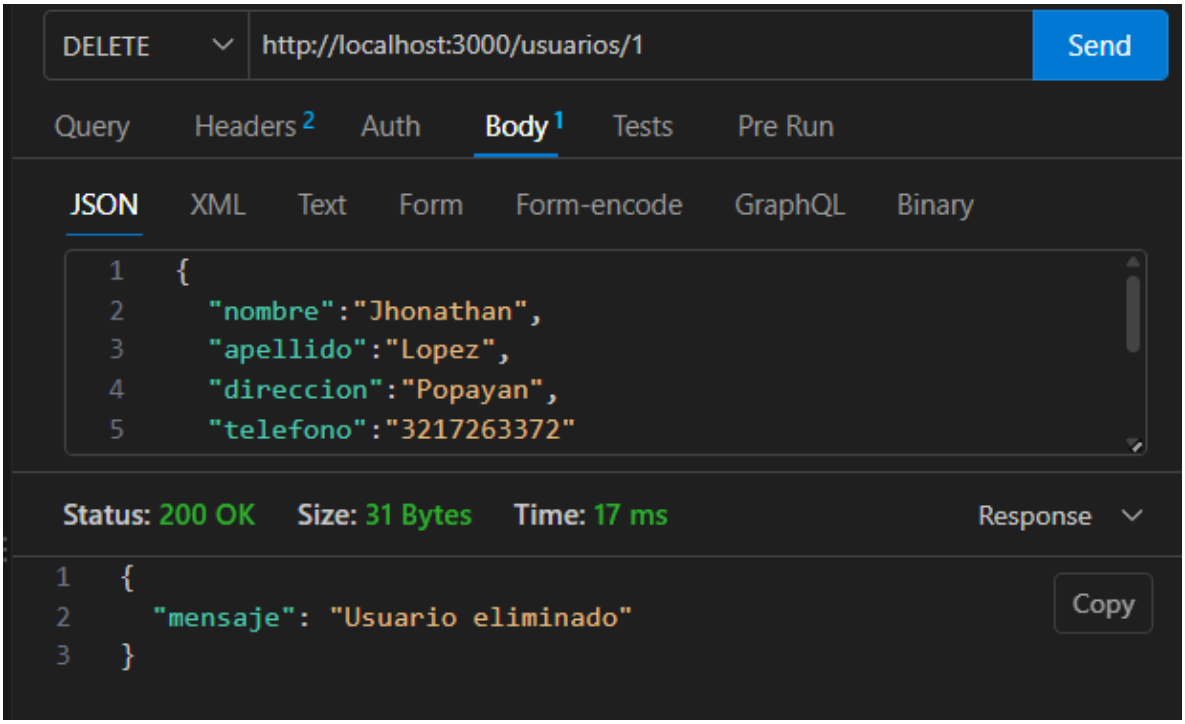
Evidencia 4:

Se modificaron los datos de un usuario existente mediante una solicitud PUT



Evidencia 5:

Se eliminó un usuario utilizando su identificador, la API devolvió un mensaje confirmando la operación.



4. Desarrollo del módulo de productos

Se implementó la clase ProductosController para administrar los productos de la empresa.

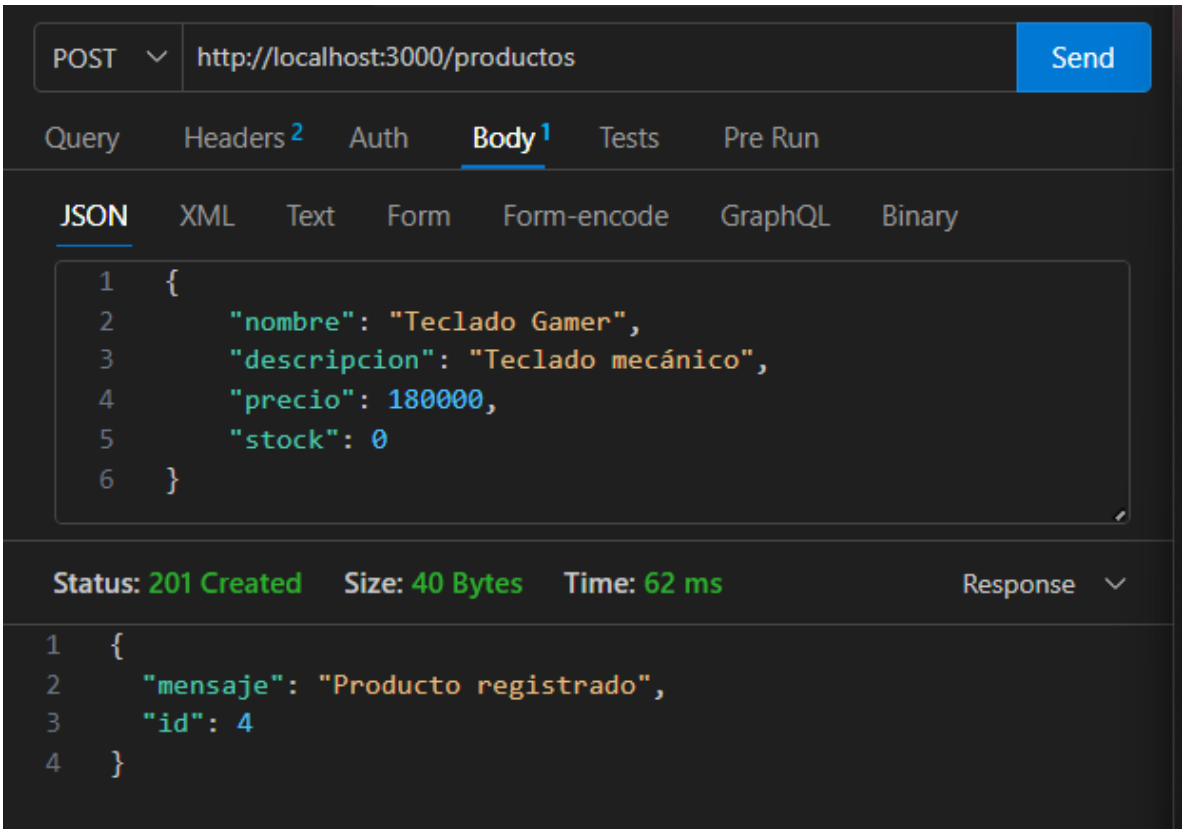
El controlador contiene siete métodos:

- guardar: registra productos.
- traer: consulta todos los productos.
- buscar: busca un producto por ID.
- actualizar: modifica los datos de un producto.
- eliminar: elimina un producto.
- agotados: consulta productos cuyo stock es igual a cero.
- bajoStock: consulta productos cuyo stock es inferior a cinco unidades.

También se implementaron validaciones para comprobar que el nombre sea obligatorio, que el precio sea numérico y que el stock sea un número entero no negativo.

Evidencia 1:

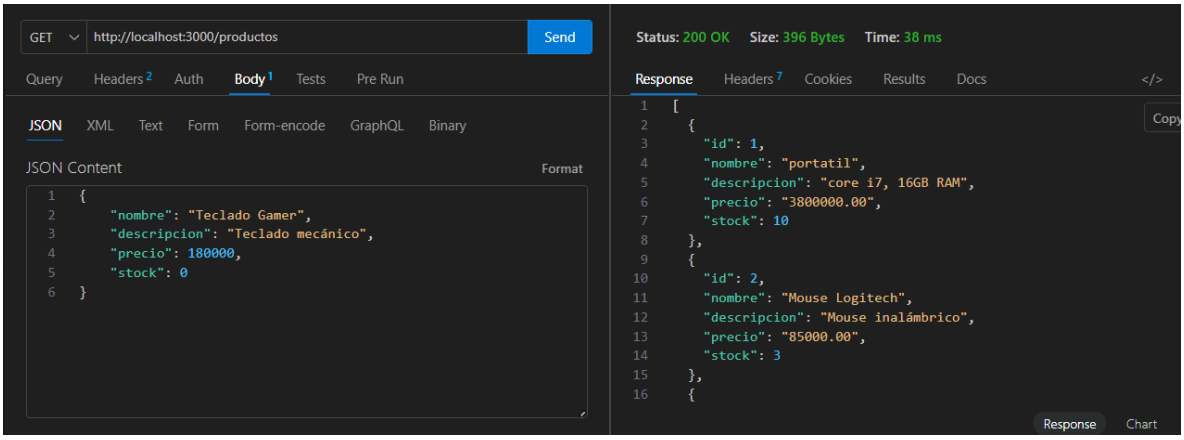
registró un producto enviando su nombre, descripción, precio y stock. La API confirmó el registro y devolvió el identificador del producto.



Evidencia 2:

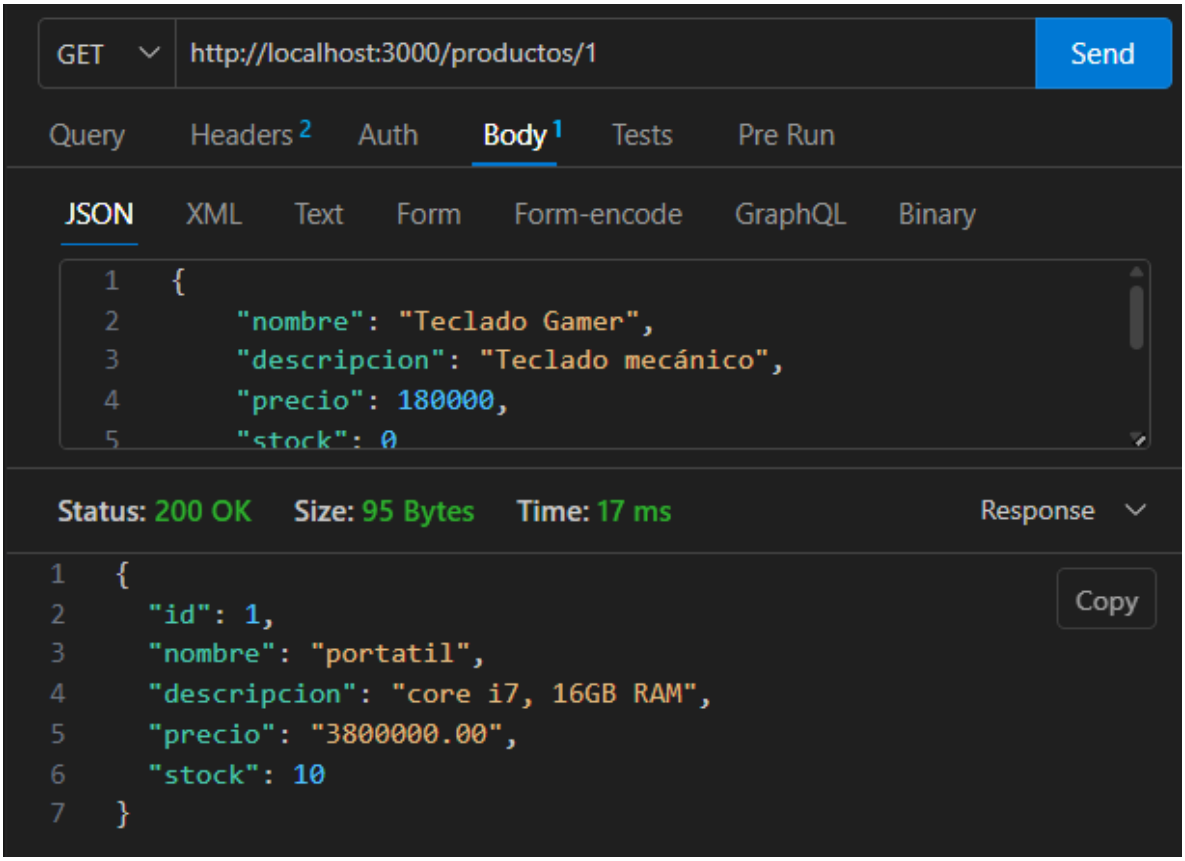
Consultelos productos almacenados en la base de datos, verificando que los registros se hayan guardaran correctamente.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Evidencia 3:

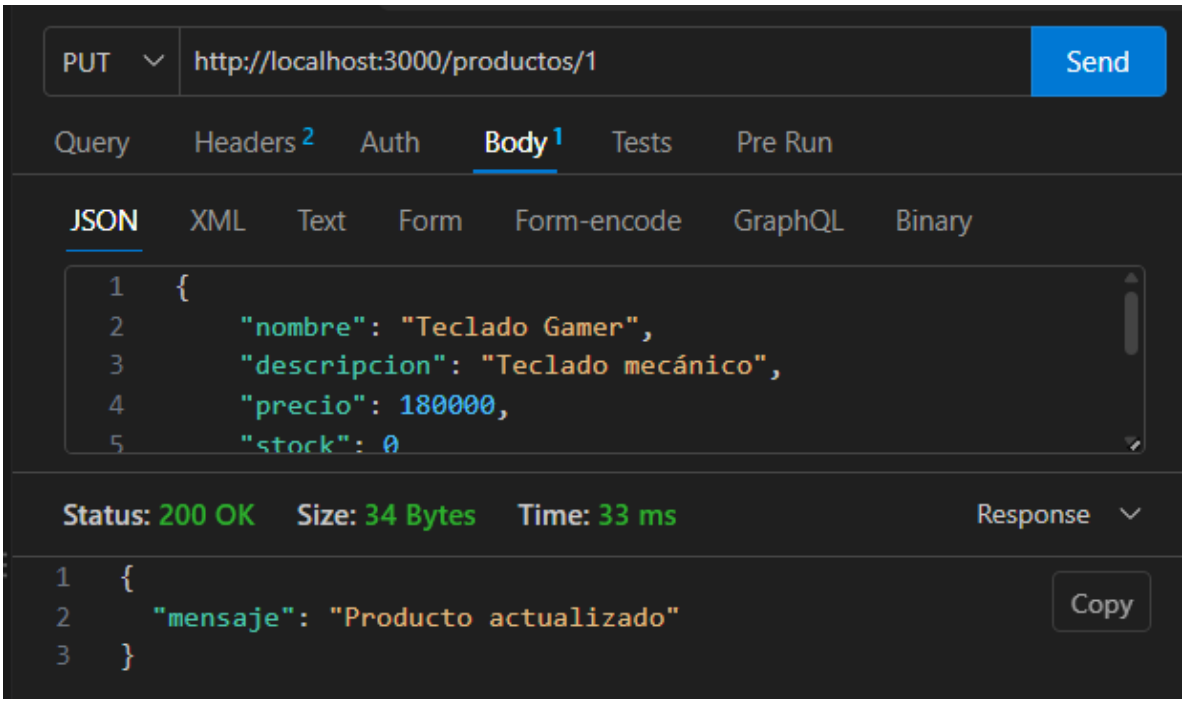
Busque un producto mediante su identificador y se obtuvo la información correspondiente.



Evidencia 4:

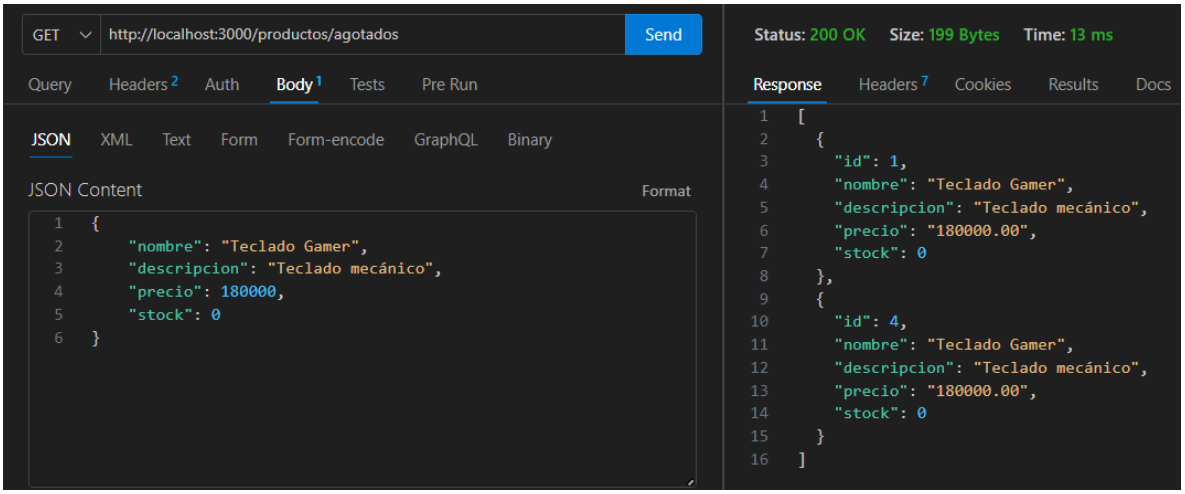
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Modifique la información de un producto existente, actualizando su descripción, precio o cantidad disponible.



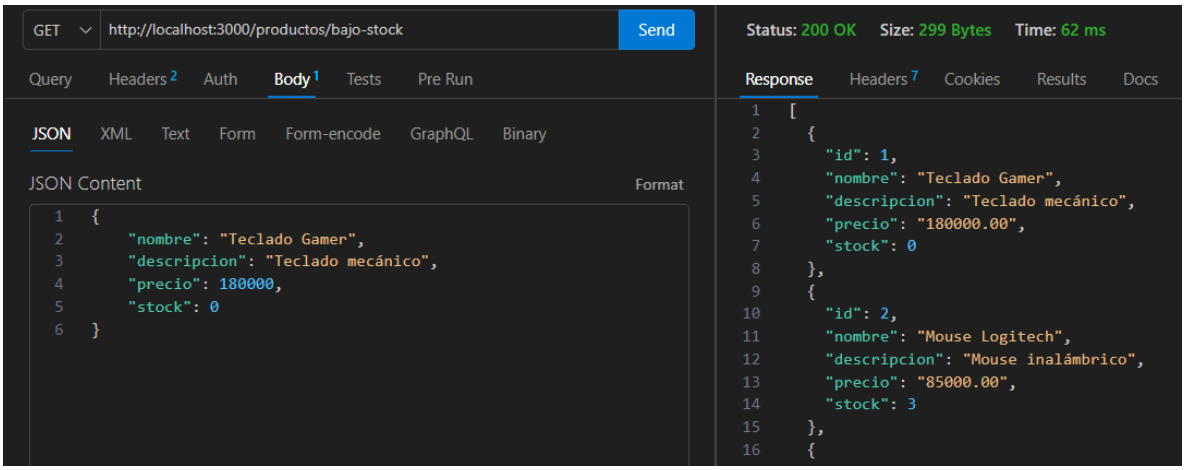
Evidencia 5:

Realice una consulta para identificar los productos cuyo stock es igual a cero.



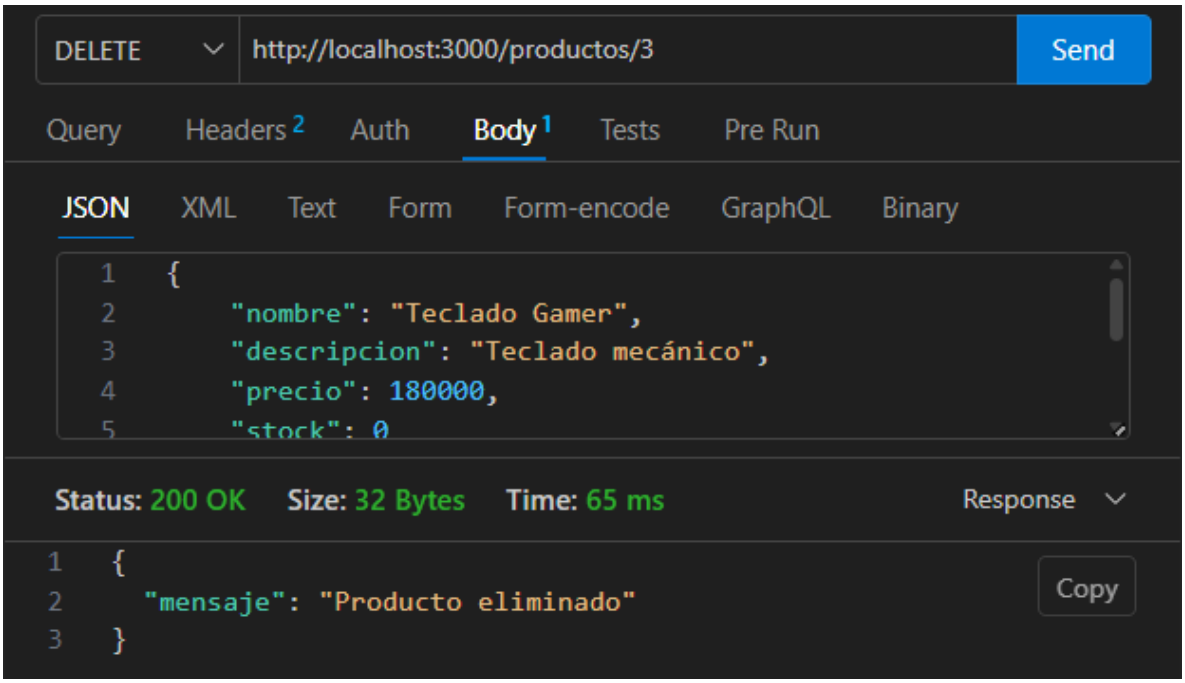
Evidencia 6:

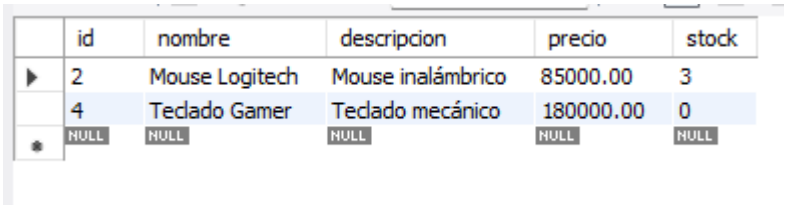
Consultelos productos que tienen menos de cinco unidades disponibles.



Evidencia 7:

Elimine un producto utilizando su identificador y se verificó que la API devolviera una respuesta de confirmación.





id	nombre	descripcion	precio	stock
2	Mouse Logitech	Mouse inalámbrico	85000.00	3
4	Teclado Gamer	Teclado mecánico	180000.00	0
NULL	NULL	NULL	NULL	NULL

5. Integracion de rutas y conexión con MySQL

Se integraron los módulos de prueba, usuarios y productos en el archivo principal app.js.

La conexión con MySQL se centralizó en este archivo y se compartió con los controladores de usuarios y productos.

Se utilizaron los métodos require() y module.exports para importar y exportar los archivos del proyecto.

Evidencia 1:

Aqui importamos los diferentes archivos de rutas y cómo se comparte la conexión con MySQL para permitir que los controladores realicen las consultas a la base de datos.

app.js > conexion

```
let express=require("express")
let cors=require("cors")
let mysql=require("mysql2")

let app = express()
app.use(express.json())
app.use(cors())

let conexion = mysql.createConnection({
    host: "127.0.0.1",
    user: "root",
    password: "adso3312932",
    database:"taller_api",
    port:3306
})
conexion.connect((error)=>{
    if(error){
        console.log("Error al conectar con MySQL:", error.message)
        return
    }
    console.log("Conexion a MySQL exitosa")
})
conexion.on("error", (error)=>{
    console.log("Error en la conexion MySQL:", error.code)
})
let PruebaRoutes = require("./routes/PruebaRoutes");
let UsuariosRoutes = require("./routes/UsuariosRoutes")(conexion);
let ProductosRoutes = require("./routes/ProductosRoutes")(conexion);

app.use("/prueba", PruebaRoutes);
app.use("/usuarios", UsuariosRoutes);
app.use("/productos", ProductosRoutes);

app.listen(3000, ()=>{
    console.log("Servidor activo en http://localhost:3000")
})
```

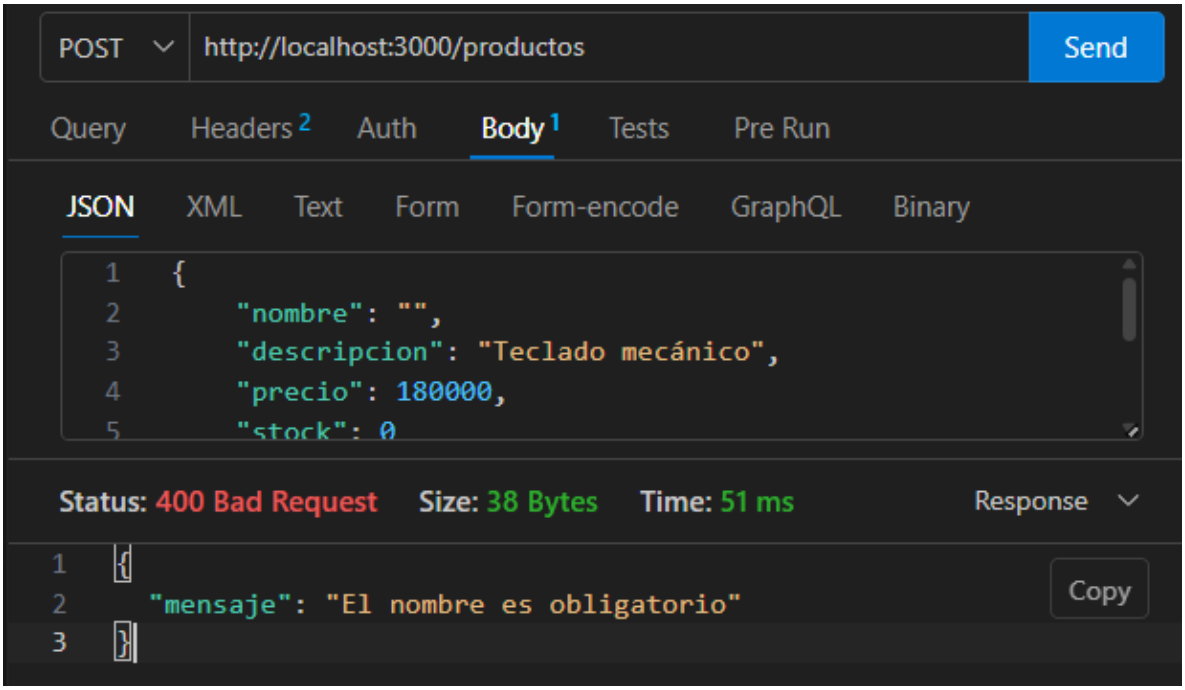
6. Identificación y solución de errores

Durante el desarrollo realice pruebas para identificar errores que presente al construir la API REST.

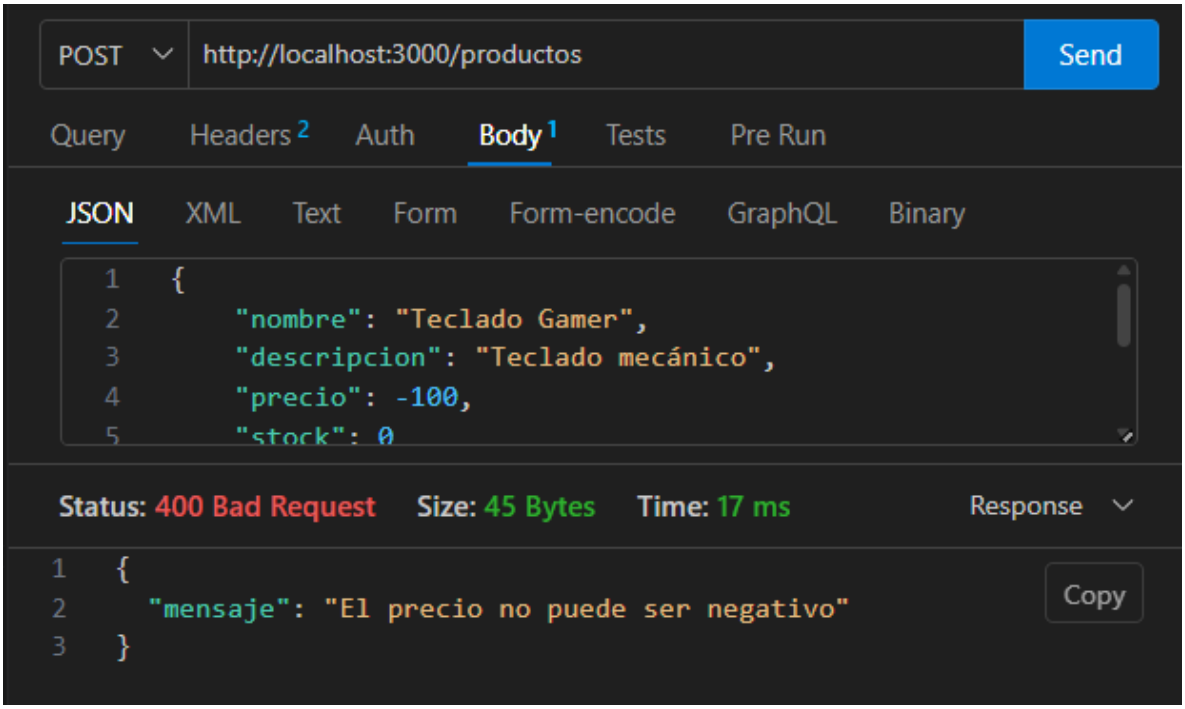
Error presentado	Causa	Solución
Cannot find module	Módulo o ruta incorrecta	Instalar el módulo o corregir la ruta
req.body undefined	Falta configurar express.json()	Agregar el middleware

Error de conexión MySQL	Problemas de conexión o credenciales	Revisar la configuración
ER_NO_SUCH_TABLE	Nombre de tabla incorrecto	Corregir el nombre de la tabla
Producto sin nombre	Falta un campo obligatorio	Agregar validación
Precio negativo	Valor no permitido	Validar que el precio no sea negativo
Stock negativo	Cantidad no válida	Validar que el stock sea entero y no negativo
ID inexistente	No se encuentra el registro	Devolver estado 404

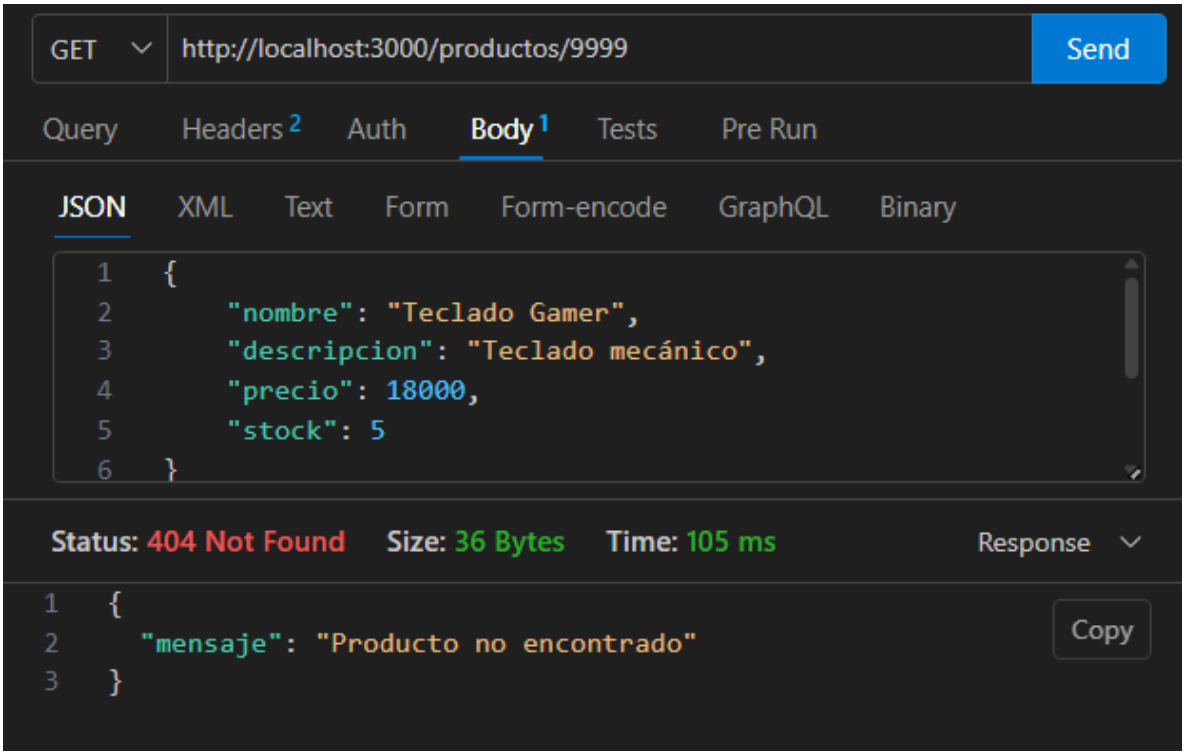
Error 1:



Error 2:


POST
http://localhost:3000/productos
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
{
"nombre": "Teclado Gamer",
"descripcion": "Teclado mecánico",
"precio": -100,
"stock": 0
Status: 400 Bad Request
Size: 45 Bytes
Time: 17 ms
Response
{
"mensaje": "El precio no puede ser negativo"
}
Copy

Error 3:


GET
http://localhost:3000/productos/9999
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
{
"nombre": "Teclado Gamer",
"descripcion": "Teclado mecánico",
"precio": 18000,
"stock": 5
}
Status: 404 Not Found
Size: 36 Bytes
Time: 105 ms
Response
{
"mensaje": "Producto no encontrado"
}
Copy

7. Pruebas integrales con Postman

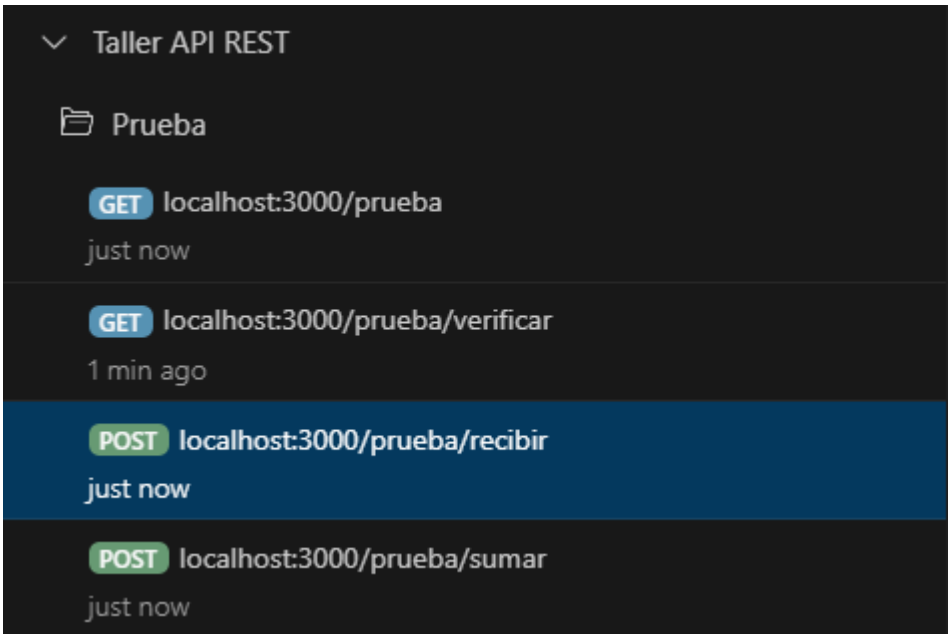
Se realizaron pruebas de los endpoints correspondientes a los módulos de prueba, usuarios y productos.

Se utilizaron los métodos HTTP GET, POST, PUT y DELETE para comprobar las operaciones de consulta, registro, actualización y eliminación.

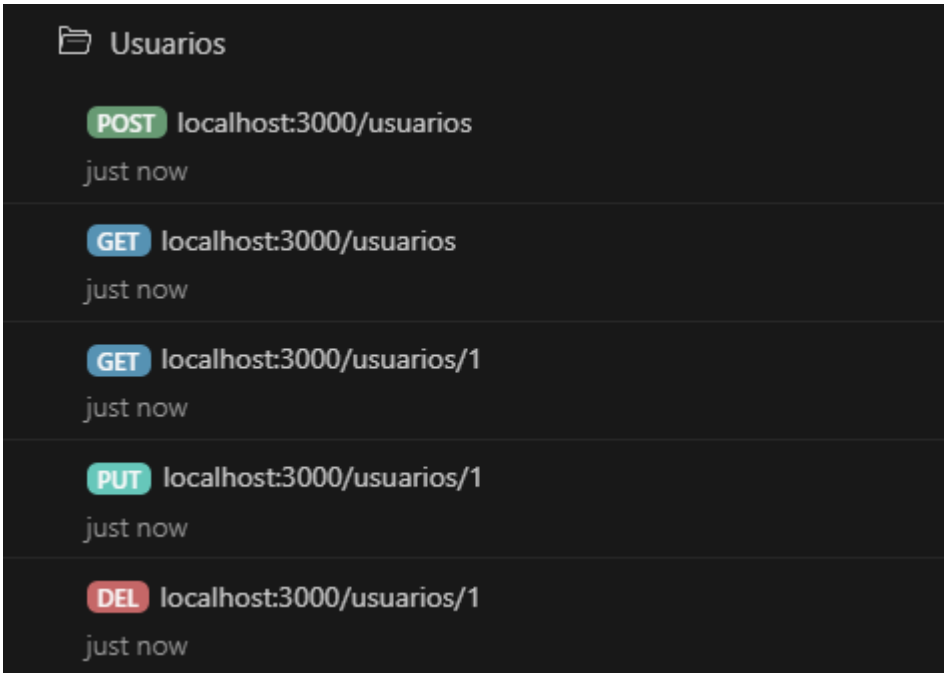
Las pruebas permitieron verificar que la API se comunica correctamente con la base de datos y devuelve respuestas según el resultado de cada solicitud.

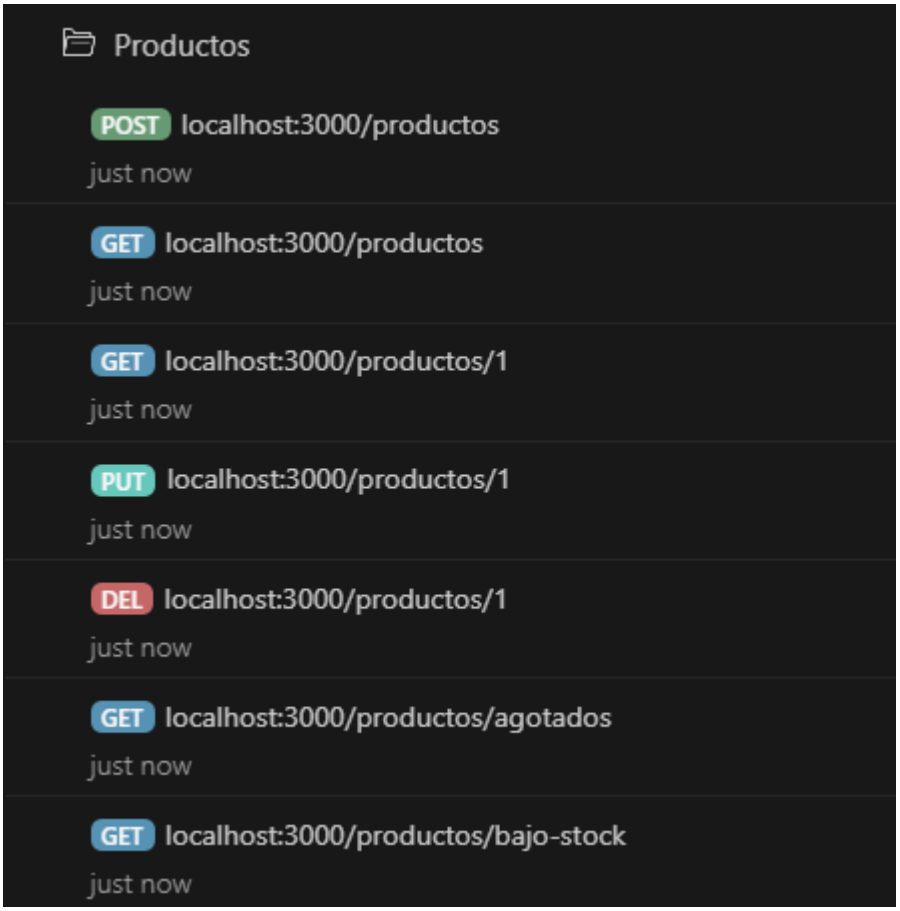
Evidencia 1:

Organice las solicitudes de Postman en una colección para facilitar la ejecución y verificación de los diferentes endpoints.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS





8. Preguntas de análisis y sustentanción

8.1. ¿Qué es Node.js y cuál es su función en el proyecto?

Node.js es un entorno que permite ejecutar JavaScript fuera del navegador. En este proyecto se utiliza para ejecutar el servidor y permitir la comunicación con la base de datos.

8.2. ¿Qué es Express y qué ventajas ofrece?

Express es un framework de Node.js que facilita la creación de servidores y rutas. Permite organizar mejor el código y gestionar las solicitudes HTTP.

8.3. ¿Qué es una API REST?

Es una interfaz que permite la comunicación entre aplicaciones mediante solicitudes HTTP para consultar, registrar, actualizar y eliminar información.

8.4. ¿Qué diferencia existe entre GET, POST, PUT y DELETE?

- GET: permite consultar información.
- POST: permite registrar información.
- PUT: permite actualizar información.
- DELETE: permite eliminar información.

8.5. ¿Qué función cumple express.json()?

Es un middleware que permite a Express interpretar los datos que llegan en formato JSON en el cuerpo de las solicitudes.

8.6. ¿Qué es una clase en JavaScript?

Es una estructura que funciona como una plantilla para crear objetos, definiendo sus propiedades y métodos.

8.7. ¿Qué es un objeto y cómo se instancia?

Un objeto es una instancia de una clase que contiene propiedades y métodos. Se instancia utilizando la palabra reservada new.

8.8. ¿Qué función cumple el constructor?

Es un método especial de una clase que se ejecuta al crear un objeto y permite inicializar sus propiedades.

8.9. ¿Qué diferencia existe entre un método tradicional y una propiedad con función flecha?

Un método tradicional tiene su propio comportamiento de this, mientras que una función flecha hereda el this del contexto donde fue creada. En el proyecto se utilizan funciones flecha para mantener correctamente el contexto de los controladores.

8.10. ¿Para qué se utiliza module.exports?

Se utiliza para exportar clases, funciones u objetos de un archivo y permitir que puedan utilizarse en otros archivos del proyecto.

8.11. ¿Qué función cumple require()?

Permite importar módulos o archivos para utilizarlos dentro de otro archivo JavaScript.

8.12. ¿Qué responsabilidad tiene un Controller?

Se encarga de procesar la lógica de las solicitudes, validar los datos, realizar las consultas a MySQL y devolver las respuestas.

8.13. ¿Qué responsabilidad tiene Routes?

Se encarga de definir las rutas y los métodos HTTP que se utilizan para dirigir las solicitudes al Controller correspondiente.

8.14. ¿Por qué la conexión MySQL se comparte desde app.js?

Porque permite centralizar la conexión y compartirla con los diferentes controladores, evitando crear una conexión independiente para cada módulo.

8.15. ¿Qué función cumplen los signos ? en las consultas SQL parametrizadas?

Son marcadores de posición que permiten enviar los valores de forma separada a la consulta SQL. Ayudan a evitar la inyección SQL y a realizar consultas más seguras.

8.16. ¿Por qué es importante controlar los errores en una API?

Porque permite identificar los problemas que se presentan, evitar fallos inesperados y devolver respuestas adecuadas al cliente, como los códigos HTTP 400, 404 y 500.

Conclusion

En conclusion el desarrollo del taller me permitió comprender cómo se conecta Postman, Express, los Routes, los Controllers y MySQL para construir una API REST funcional, logrando desarrollar una API REST utilizando Node.js, Express y MySQL, tambien implementando los controladores y rutas para organizar el proyecto, tambien para realizar operaciones CRUD para administrar usuarios y productos, utilice consultas SQL parametrizadas para interactuar con la base de datos, realicepruebas con Postman para verificar el funcionamiento de los endpoints e identifique errores para asi poder aplicar soluciones mediante validaciones y manejo de respuestas HTTP.